



# **University Institute of Engineering**

## **Department of Computer Science & Engineering**

### **COMPUTER ORGANIZATION & ARCHITECTURE**

#### **(23CST-204/23ITT-204)**

**ER. SHIKHA ATWAL**

**E11186**

**ASSISTANT PROFESSOR**

**BE-CSE**

## INPUT-OUTPUT INTERFACE

- The method that is used to transfer information between internal storage and external I/O devices is known as I/O interface.
- The CPU is interfaced using special communication links by the peripherals connected to any computer system. These communication links are used to resolve the differences between CPU and peripheral.
- There exist special hardware components between CPU and peripherals to supervise and synchronize all the input and output transfers that are called interface units.

## Modes of Transfer

The binary information that is received from an external device is usually stored in the memory unit. The information that is transferred from the CPU to the external device is originated from the memory unit. CPU merely processes the information but the source and target are always the memory unit. Data transfer between CPU and the I/O devices may be done in different modes.

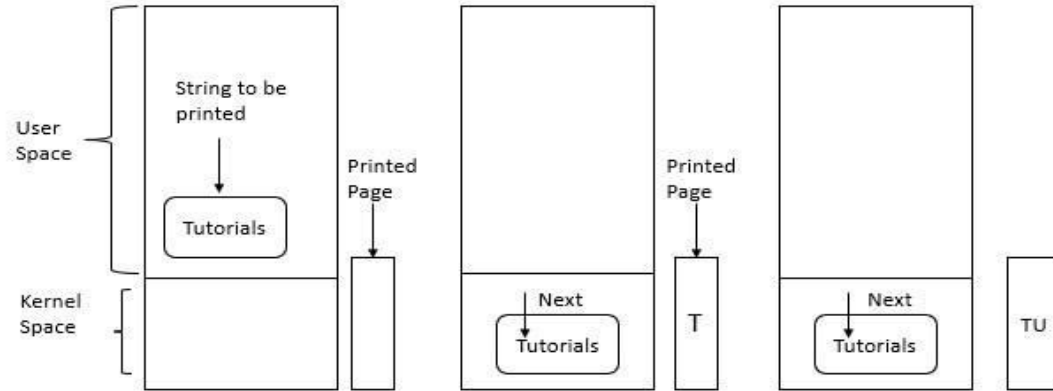
Data transfer to and from the peripherals may be done in any of the three possible ways

1. Programmed I/O
2. Interrupt-initiated I/O
3. Direct memory access (DMA)

## 1. Programmed I/O

It is one of the simplest forms of I/O where the CPU has to do all the work. It is due to the result of the I/O instructions that are written in the computer program. Each data item transfer is initiated by an instruction in the program. Usually, the transfer is from a CPU register and memory. In this case it requires constant monitoring by the CPU of the peripheral devices. This technique is called Programmed I/O.

Consider a user process that wants to print the Nine-character string “TUTORIALS” on the printer with the help of a serial interface. The software first assembles the string in a buffer in user space, as shown in the figure below.



**Step 1** – The user process acquires the printer for writing by using a system call to open it.

**Step 2** – If the printer is currently in use by another process, this system call fails and returns an error code or it blocks until the printer is available, depending on the operating system and the parameters of the call.

**Step 3** – Once the printer is available, the user process makes a system call telling the operating system to print the string on the printer.

**Step 4** – The operating system generally copies the buffer with the string to an array.

**Step 5** – It then checks to see if the printer is currently available. If not, it waits until it is. Whenever the printer is available, the operating system copies the first character to the printer's data register, In the above example use memory-mapped I/O. This action activates the printer. The character does not appear still, because some printers buffer a line or a page before printing anything.

**Step 6** – In the next figure, we see that the first character has been printed and that the system has marked the “U” as the next character to be printed.

**Step 7** – Whenever it has copied the first character to the printer, the operating system checks to see if the printer is ready to accept another one.

**Step 8** – Generally, the printer has a second register, which gives its status. The act of writing to the data register causes the status to become not ready.

**Step 9** – When the printer controller has processed the current character, it indicates its availability by setting some bit in its status register or putting some value in it.

**Step 10** – At this point the operating system waits for the printer to become ready again.

**Step 11** – It prints the next character, as shown in the third figure.

**Step 12** – This loop continues till the entire string has been printed.

**Step 13** – Then control returns to the user process.

Programmable I/O is one of the I/O technique other than the interrupt-driven I/O and direct memory access (DMA). The programmed I/O was the simplest type of I/O technique for the exchanges of data or any types of communication between the processor and the external devices. With programmed I/O, data are exchanged between the processor and the I/O module. The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data. When the processor issues a command to the I/O module, it must wait until the I/O operation is complete. If the processor is faster than the I/O module, this is wasteful of processor time.

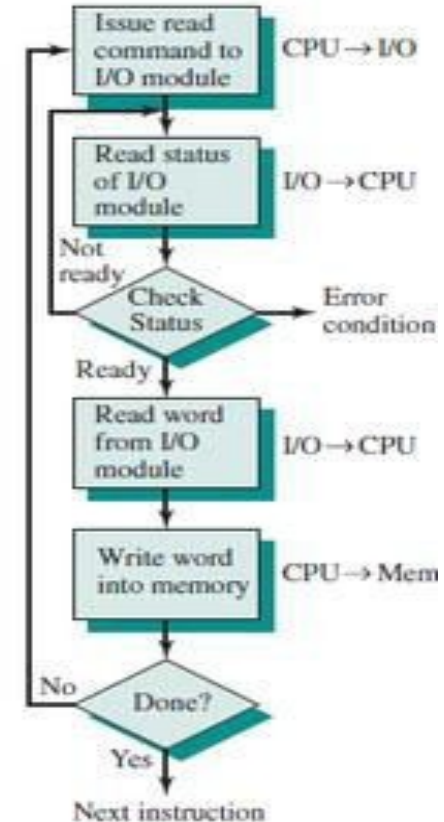


The overall operation of the programmed I/O can be summaries as follows:

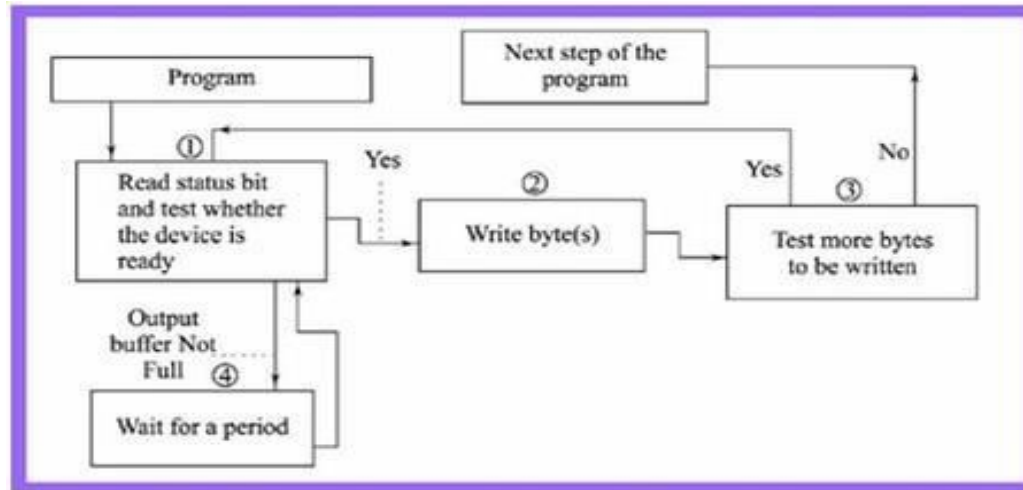
- 1.The processor is executing a program and encounters an instruction relating to I/O operation.
- 2.The processor then executes that instruction by issuing a command to the appropriate I/O module.
- 3.The I/O module will perform the requested action based on the I/O command issued by the processor (READ/WRITE) and set the appropriate bits in the I/O status register.
- 4.The processor will periodically check the status of the I/O module until it finds that the operation is complete.

## Programmed I/O Mode Input Data Transfer

1. Each input is read after first testing whether the device is ready with the input (a state reflected by a bit in a status register).
2. The program waits for the ready status by repeatedly testing the status bit and till all targeted bytes are read from the input device.
3. The program is in busy (non-waiting) state only after the device gets ready else in wait state.



## Programmed I/O Mode Output Data Transfer



1. Each output written after first testing whether the device is ready to accept the byte at its output register or output buffer is empty.
2. The program waits for the ready status by repeatedly testing the status bit(s) and till all the targeted bytes are written to the device.
3. The program in busy (non-waiting) state only after the device gets ready else wait state.

## I/O Commands

To execute an I/O-related instruction, the processor issues an address, specifying the particular I/O module and external device, and an I/O command. There are four types of I/O commands that an I/O module may receive when it is addressed by a processor:

- **Control:** Used to activate a peripheral and tell it what to do. For example, a magnetic- tape unit may be instructed to rewind or to move forward one record. These commands are tailored to the particular type of peripheral device.
- **Test:** Used to test various status conditions associated with an I/O module and its peripherals. The processor will want to know that the peripheral of interest is powered on and available for use. It will also want to know if the most recent I/O operation is completed and if any errors occurred.

- **Read:** Causes the I/O module to obtain an item of data from the peripheral and place it in an internal buffer. The processor can then obtain the data item by requesting that the I/O module place it on the data bus.
- **Write:** Causes the I/O module to take an item of data (byte or word) from the data bus and subsequently transmit that data item to the peripheral.

## I/O Instruction

With programmed I/O, there is a close correspondence between the I/O-related instructions that the processor fetches from memory and the I/O commands that the processor issues to an I/O module to execute the instructions. Typically, there will be many I/O devices connected through I/O modules to the system. Each device is given a unique identifier or address. When the processor issues an I/O command, the command contains the address of the desired device. Thus, the I/O module must interpret the address lines to determine if the command is for itself and also which external devices that the address refer to.

When the processor, main memory share a common bus, two modes of addressing are possible:

1. Memory mapped I/O
2. Isolated I/O

With **memory mapped I/O**, there is a single address space for memory locations and I/O devices and the processor treats the status and data registers of I/O modules as memory locations and uses the same machine instructions to access both memory and I/O devices. So, for example, with 10 address lines, a combined total of  $2^{10} = 1024$  memory locations and I/O addresses can be supported, in any combination. With memory-mapped I/O, a single read line and a single write line are needed on the bus.



With **isolated I/O**, the bus may be equipped with memory read and write plus input and output command lines. Now, the command line specifies whether the address refers to a memory location or an I/O device. The full range of addresses may be available for both. Again, with 10 address lines, the system may now support both 1024 memory locations and 1024 I/O addresses.

For most types of processors, there is a relatively large set of different instructions for referencing memory. If isolated I/O is used, there are only a few I/O instructions. Thus, an advantage of memory-mapped I/O is that this large repertoire of instructions can be used, allowing more efficient programming. A disadvantage is that valuable memory address space is used up.

## Differences between Isolated I/O and Memory Mapped I/O:

| Isolated I/O                                                    | Memory Mapped I/O                                                            |
|-----------------------------------------------------------------|------------------------------------------------------------------------------|
| Isolated I/O uses separate memory space.                        | Memory mapped I/O uses memory from the main memory.                          |
| Limited instructions can be used. Those are IN, OUT, INS, OUTS. | Any instruction which references to memory can be used.                      |
| The address for Isolated I/O devices are called ports           | Memory mapped I/O devices are treated as memory locations on the memory map. |

## **Advantages of Programmed I/O**

- Simple to implement
- Very little hardware support

## **Disadvantages of Programmed I/O**

- Busy in waiting for completion of task
- Ties up CPU for long period with no useful work

## References

### Reference Books:

- J.P. Hayes, “Computer Architecture and Organization”, Third Edition.
- Mano, M., “Computer System Architecture”, Third Edition, Prentice Hall.
- Stallings, W., “Computer Organization and Architecture”, Eighth Edition, Pearson Education.

### Text Books:

- Carpinelli J.D,” Computer systems organization &Architecture”, Fourth Edition, Addison Wesley.
- Patterson and Hennessy, “Computer Architecture”, Fifth Edition Morgan Kaufman.

## Other References

- [Programmed I/O \(brainkart.com\)](https://brainkart.com)
- <https://youtu.be/CilIrpB2bs?si=9I2K5yX3iFCAZ4eF>
- <https://youtu.be/ZC7CedalOYE?si=6vWuju9FjjeIhMcM>